



**ПОСТРОЕНИЕ КАЧЕСТВЕННЫХ
JAVA-ПРИЛОЖЕНИЙ ЧЕРЕЗ
МОДУЛЬНОЕ ТЕСТИРОВАНИЕ.**

**АБАКУМОВА МАРИЯ
ДОРЖИЕВА АРЮНА**

АННОТАЦИИ УПРАВЛЕНИЯ ТЕСТОМ

- `@Test` – основной маркер метода, который должен быть запущен как тест.
- `@BeforeEach` – код для подготовки окружения (запускается перед каждым тестом).
- `@AfterEach` – код для очистки ресурсов (запускается после каждого теста).
- `@BeforeAll` – глобальная настройка для всего класса (запускается один раз).
- `@AfterAll` – завершение работы и закрытие тяжелых ресурсов.
- `@DisplayName("...")` – понятное описание теста на любом языке.
- `@Disabled` – пропуск теста.

УТВЕРЖДЕНИЯ И РЕЗУЛЬТАТЫ

3

- `assertEquals` – проверка равенства ожидаемого и фактического значений.
- `assertNotEquals` – проверка, что значения разные.
- `assertSame` – проверка, что переменные ссылаются на один и тот же объект в памяти.
- `assertTrue` / `assertFalse` – проверка логических условий.
- `assertNull` / `assertNotNull` – проверка инициализации объекта.
- `assertThrows` – подтверждение того, что код выбрасывает нужное исключение при ошибке.
- `assertAll` – группировка нескольких проверок: тест не остановится, пока не выполнит их все (удобно для полной проверки объекта).



ПРИМЕР КОДА

НАПИСАНИЕ АННОТАЦИЙ

5

```
@BeforeEach
void init() {
    service = new OrderService();
    System.out.println("Тест начат: объект Service создан");
}

@AfterEach
void cleanUp() {
    repository.deleteAll();
    System.out.println("Тест окончен: база очищена");
}
```

НАПИСАНИЕ ПРОСТОГО UNIT-ТЕСТА

```
@Test
@DisplayName("1. Базовая проверка: список заказов пуст")
void testEmpty() {
    assertTrue(service.getAllOrders().isEmpty(), "Изначально список должен быть пуст");
}
```

```
@Test
@DisplayName("2. Проверка чисел: сложение double")
void shouldCalculateTotal() {
    double result = calculator.sum(10.5, 20.0);
    assertEquals(30.5, result, 0.001);
}
```

```
@Test
@DisplayName("3. Проверка на Null: создание пользователя")
void shouldCreateUser() {
    User user = repository.save(new User("Alice", "Admin", true));
    assertNotNull(user, "Объект не должен быть null");
    assertNotNull(user.getId(), "ID должен быть сгенерирован");
}
```

```
@Test
@DisplayName("4. Проверка исключений: отрицательная цена")
void shouldFailOnNegativePrice() {
    assertThrows(IllegalArgumentException.class, () -> {
        product.setPrice(-100);
    }, "Должно быть выброшено исключение при цене < 0");
}
```

```
@Test
@DisplayName("5. Групповая проверка (assertAll): профиль пользователя")
void shouldUpdateProfile() {
    User user = new User("Oleg", "Dev", true);

    assertAll("Проверка всех полей объекта сразу",
        () -> assertEquals("Oleg", user.getName()),
        () -> assertEquals("Dev", user.getJob()),
        () -> assertTrue(user.isActive())
    );
}

@Test
@Disabled("Функционал оплаты не разработан")
@DisplayName("6. Отключенный тест")
void testCryptoPayment() {
    fail("Этот тест не должен был запуститься!");
}
```


JUNIT 6

JUnit 6 – это современное обновление популярного фреймворка для тестирования Java-кода, выпущенное в октябре 2025 года.

Основные преимущества JUnit 6:

- Переход на современные стандарты Java. Для работы теперь требуется минимум Java 17.
- Безопасность работы с null. Внедрена поддержка библиотеки JSpecify.
- Чистота и порядок. Из фреймворка удалили множество устаревших функций (API), которые копились годами.
- Единые версии компонентов. Раньше разные части JUnit имели разные номера версий, что путало при настройке.
- Понятные отчеты. Улучшено отображение имен в параметризованных тестах.

ПРОДВИНУТЫЕ ВОЗМОЖНОСТИ JUNIT

Параметризованные тесты (`@ParameterizedTest`) – позволяют запускать один и тот же тест с разными наборами данных.

`@ValueSource` – простой набор данных (используется, когда нужно проверить однотипные значения).

`@CsvSource` – табличные данные.

`@CsvFileSource` – внешние файлы данных.

```
@ParameterizedTest
@ValueSource(strings = {"12345", "qwerty", "admin", "password"})
@DisplayName("7. Проверка слишком коротких паролей")
void testShortPasswords(String password) {
    assertFalse(PasswordValidator.isValid(password), "Пароль '" + password + "' не короткий");
}

@ParameterizedTest
@CsvSource({
    "10, 20, 30",
    "0, 5, 5",
    "-1, 1, 0"
})
@DisplayName("8. Параметризованный тест: сложение")
void testAddition(int a, int b, int expected) {
    assertEquals(expected, a + b);
}
```

СПАСИБО ЗА ВНИМАНИЕ